

Aula 21: Introdução ao Machine Learning Clássico

Algoritmos do Scikit-Learn, Hiperparâmetros e Isolamento de Dados

Extração e Preparação de Dados (IBM8915)

Professor: Luís Aramis
IBMEC

O Paradigma do Machine Learning

Diferente da programação tradicional, onde escrevemos regras lógicas **manuais**, no ML o algoritmo aprende padrões ocultos **direto dos dados**.

Principais Divisões:

- ▶ **Supervisionado:** Temos a "resposta" (rótulo) no histórico.
 - ▶ **Classificação:** Prever uma categoria (ex: Sim/Não, Risco Alto/Baixo).
 - ▶ **Regressão:** Prever um número contínuo (ex: Preço de um imóvel).
- ▶ **Não Supervisionado:** O modelo descobre padrões sem nenhum gabarito prévio (ex: Segmentação/Agrupamento de Clientes).

O Isolamento Sagrado: Treino vs Teste

Antes de criar qualquer modelo, a regra fundamental é isolar parte dos dados. Se o modelo estudar e for avaliado com as mesmas provas, ele apenas memoriza o resultado (*Overfitting*).

```
1 from sklearn.model_selection import train_test_split
2
3 # Separando 80% dos dados para treino e 20% para teste (reserva isolada)
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.20, random_state=42
6 )
7
```

- ▶ **X_train / y_train:** O laboratório onde o modelo extrai as regras.
- ▶ **X_test / y_test:** A prova final, cega e inédita.

Apesar do nome, é um modelo linear estatístico usado primariamente para **classificação binária** (0 ou 1).

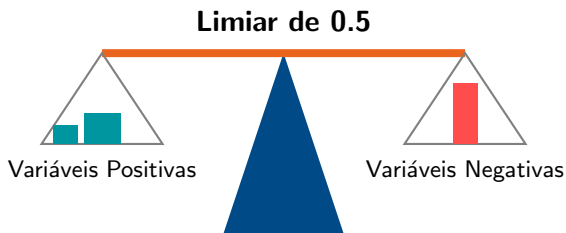
Como funciona? Calcula a probabilidade matemática mapeando combinações lineares numa **Curva Sigmoide**.

$$P(y = 1|z) = \frac{1}{1 + e^{-z}}$$

Por que escolher? É altamente **interpretável**. Os coeficientes (*odds ratio*) explicam exatamente qual variável puxou o resultado para cima ou para baixo, sendo ideal para cenários altamente regulamentados (ex: finanças).

Analogia: A Balança de Julgamento

Imagine um juiz pesando evidências. As variáveis são os "pesos" nos pratos da balança. O modelo soma os pesos a favor da condenação e da inocência. O *threshold* (limiar) de 0.5 dita quando a balança tomba definitivamente para um lado.



Regressão Logística: Código

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Instancia o modelo definindo um threshold de regularizacao (C)
4 modelo_log = LogisticRegression(C=1.0, max_iter=1000)
5
6 # Treinamento do modelo
7 modelo_log.fit(X_train, y_train)
8
9 # Avaliacao padrao com a metrica de Acuracia
10 score = modelo_log.score(X_test, y_test)
11
12 # Coeficientes aprendidos (impacto explicavel de cada feature)
13 pesos_decisao = modelo_log.coef_
14 print(pesos_decisao)
15
```

Problema Prático: Credit Scoring

Aprovar ou rejeitar crédito bancário exige que o modelo acerte e consiga explicar por que rejeitou. Avaliamos a performance cruzando previsões vs realidade através da **Matriz de Confusão**.

		Previu Pagou (1)	Previu Calote (0)
Real	Calote	Falso Negativo 15	Verdadeiro Negativo 95
	Pagou	Verdadeiro Positivo 850	Falso Positivo 40

Modelos não-paramétricos que realizam partições recursivas dos dados criando fluxos lógicos de "se-então".

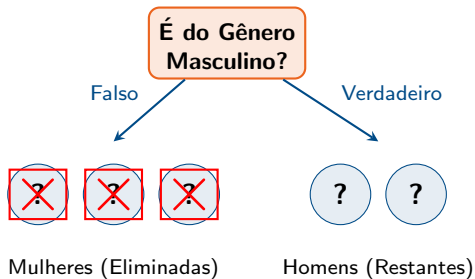
Como funciona? A cada etapa, o algoritmo tenta dividir o conjunto de dados da forma mais pura possível, utilizando cálculos como **Índice de Gini** ou **Entropia** (Maximizando o Ganho de Informação).

Por que escolher? Não exigem que os dados sigam distribuições normais, toleram dados mistos, dispensam o escalonamento (StandardScaler) e fornecem auditoria visual imediata. O risco primário é o *Overfitting* se crescerem demais.

Árvores de Decisão: Analogia

Analogia: O Jogo "Cara a Cara"

Para adivinhar o personagem rápido, não pergunte "Ele usa óculos vermelhos?". Pergunte: "Ele é homem?". A Árvore de Decisão faz a conta do Gini para achar a pergunta exata que divide o tabuleiro pela metade e elimina a maior incerteza instantaneamente.

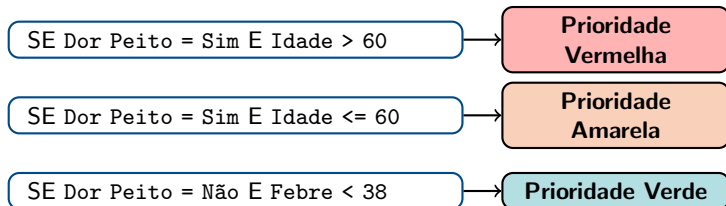


Árvores de Decisão: Código

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 import matplotlib.pyplot as plt
3
4 # Arvore limitada a profundidade 4 para evitar overfitting massivo
5 arvore = DecisionTreeClassifier(max_depth=4, criterion='gini')
6 arvore.fit(X_train, y_train)
7
8 # Funcao nativa para visualizar as regras logicas na tela
9 plt.figure(figsize=(10, 6))
10 plot_tree(arvore, filled=True, feature_names=X.columns,
11           class_names=['Reprovado', 'Aprovado'])
12 plt.show()
13
```

Problema Prático: Triagem de Pronto-Socorro

Durante um plantão, enfermeiros precisam decidir protocolos vitais em segundos. A Árvore traduz dados complexos numa **Tabela Rápida de Regras Lógicas**.



Um poderoso modelo *Ensemble* (conjunto) que não confia em uma única árvore de decisão, mas simula uma imensa floresta de centenas delas em paralelo.

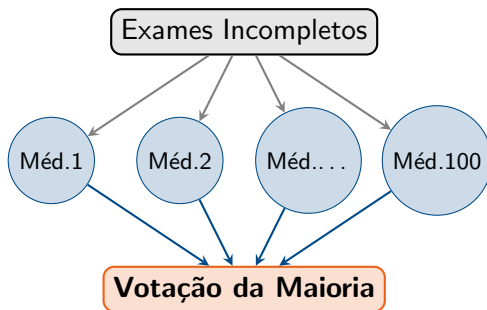
Como funciona? Utiliza **Bagging** (Bootstrap Aggregating) para entregar recortes aleatórios dos dados (linhas e colunas) para cada árvore. O resultado final sai por **votação da maioria**.

Por que escolher? (Trade-off Viés-Variância) A floresta praticamente anula o maior ponto fraco da árvore isolada (alta variância e propensão a decorar o treino). O consenso da multidão dissolve os ruídos de *outliers*, criando um modelo ultra robusto.

Random Forests: Analogia

Analogia: O Conselho Médico

Consultar um único médico pode resultar num laudo altamente enviesado por sua especialidade. Se formarmos um conselho com 100 médicos, e dermos a eles exames parciais e misturados (*Bagging*), a decisão majoritária diluirá absurdamente o erro de um médico isolado.

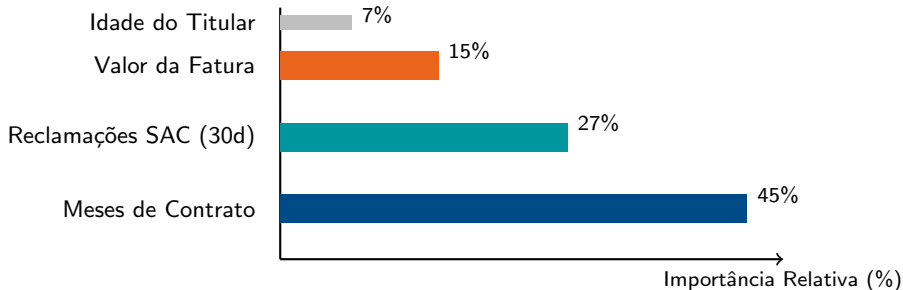


Random Forests: Código

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 # Criando a floresta com 100 arvores paralelas (n_estimators)
4 floresta = RandomForestClassifier(n_estimators=100, random_state=42)
5
6 # O treinamento distribui amostragens aleatorias internamente
7 floresta.fit(X_train, y_train)
8
9 # Extracao do maior poder da RF: A forza de cada variavel
10 importancias = floresta.feature_importances_
11
12 # (Opcional) Visualizacao ou printacao das importancias
13 print(importancias)
14
```

Problema Prático: Previsão de Churn em Telecom

A previsão de perda de clientes lida com centenas de dados de comportamento que não formam padrões lógicos simples. A Random Forest encontra as interações invisíveis e revela a **Importância Relativa das Features**.



A Sintonia Fina: Parâmetros vs Hiperparâmetros

Para que um algoritmo chegue no "estado da arte", precisamos ajustá-lo:

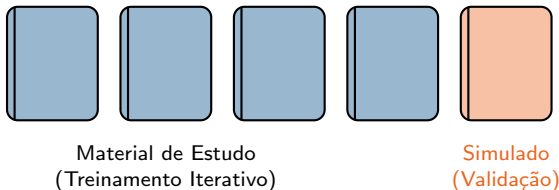
- ▶ **Parâmetros:** Valores calculados *pelo algoritmo* lendo os dados (ex: Os coeficientes calculados pela Logística).
- ▶ **Hiperparâmetros:** Configurações arquiteturais definidas *pelo Cientista de Dados* antes do treino (ex: Profundidade Máxima de uma Árvore, C da Logística, número de estimadores da Floresta).

A busca da combinação ideal nunca pode testar a acurácia na base de *Teste Oficial*, ou o modelo se enviesará. Para isso usamos a **Validação Cruzada (K-Fold)**.

Hiperparâmetros: Analogia K-Fold

Analogia: A Preparação para o ENEM

Se o aluno estuda pelas exatas provas que ele vai fazer amanhã, tirar 10 é *Overfitting* e não aprendizado. O K-Fold é isolar o banco de apostilas em 5 cadernos. Ele estuda por 4 e simula com 1, repetindo isso e revezando o caderno de simulado para provar que a regra se aplica a múltiplos cenários virgens.



A Implementação: GridSearchCV e K-Fold

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.tree import DecisionTreeClassifier
3
4 # Definimos as combinacoes possiveis de hiperparametros
5 parametros = {
6     'max_depth': [3, 5, 10, None],
7     'min_samples_split': [2, 5, 10]
8 }
9
10 # Criamos uma malha de busca com Validacao Cruzada (cv=5 folds)
11 busca = GridSearchCV(DecisionTreeClassifier(), parametros, cv=5)
12
13 # A busca eh feita *exclusivamente* no Treino!
14 busca.fit(X_train, y_train)
15
16 # O Scikit-learn ja seleciona e treina a melhor configuracao
17 print(f"Melhores hiperparametros: {busca.best_params_}")
18
```

O Perigo Silencioso: Data Leakage

Vazamento de Dados (Data Leakage) ocorre quando informações matemáticas do conjunto de Teste contaminam prematuramente o conjunto de Treino durante a fase de limpeza.

Se você aplicar uma Imputação (média global) ou Padronização (StandardScaler) sobre a base **antes do train_test_split**, os parâmetros como média estatística da base de teste se infiltrarão nos cálculos do treino. O modelo obterá nota máxima no laboratório, mas fará o banco perder dinheiro em produção.

Data Leakage: Analogia

Analogia: A Prova com Vazamento de Gabarito

O Leakage é quando o aluno consegue ouvir os sussurros de professores discutindo as respostas no corredor antes da prova. A sala de aula está "contaminada" com o futuro. O **Pipeline** é o recurso que blinda a avaliação: trancando os alunos e as provas em edifícios impenetráveis e blindados de vazamentos.

Vazamento (Leakage)

Tirar Média de TUDO



Divisão Treino/Teste

VS

Blindagem (Pipeline)

Divisão Treino/Teste



Tirar Média
só do Treino

Automação e Defesa: Pipelines

O Pipeline agrupa os transformadores lógicos, automatizando chamadas para aplicar o fit **apenas** no treino e aplicar rigorosamente só o transform na hora da prova final (teste).

```
1 from sklearn.pipeline import make_pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.impute import SimpleImputer
4 from sklearn.linear_model import LogisticRegression
5
6 # O Pipeline enfileira a ordem correta, sem interrupcoes
7 pipeline_blindado = make_pipeline(
8     SimpleImputer(strategy='median'),
9     StandardScaler(),
10    LogisticRegression(C=0.5)
11 )
12
13 # O pipeline.fit blinda o treino (as medias e desvios ficam restritos a ele)
14 pipeline_blindado.fit(X_train, y_train)
15
16 # O teste eh imaculado e apenas sofre a transformacao antes de ser inferido
17 score_real = pipeline_blindado.score(X_test, y_test)
18
```

Exercício Prático

Abra o ambiente corporativo / Jupyter Lab e acesse a pasta da **Aula 13**. Inicie a trilha contida no arquivo: `lab_13_pipelines_intro.ipynb`

Objetivos da Aula:

1. Garantir que todo processamento (`StandardScaler`, etc.) seja envelopado em `Pipelines`.
2. Substituir o `LogisticRegression` por `RandomForestClassifier` no pipeline e avaliar ganhos.
3. Parametrizar com `GridSearchCV` isoladamente o bloco de Treino.

Bom Código!